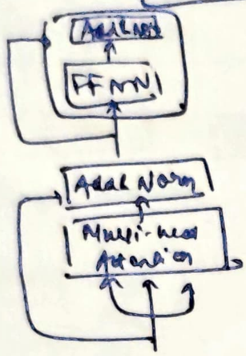
Lec-83 Decoder Architecture

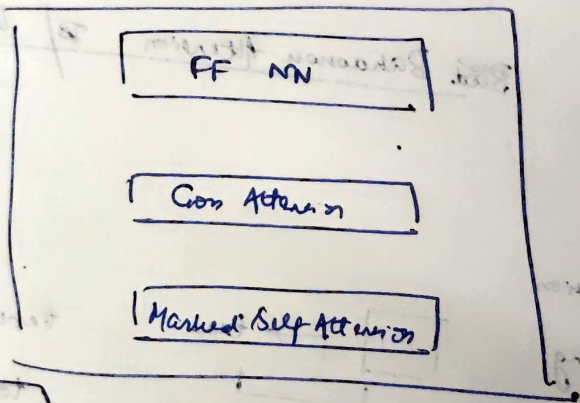
Day-69 Training and Inference

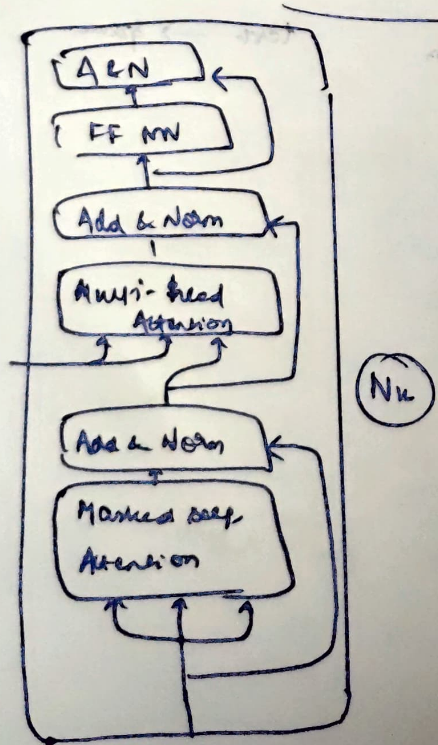
Encoder

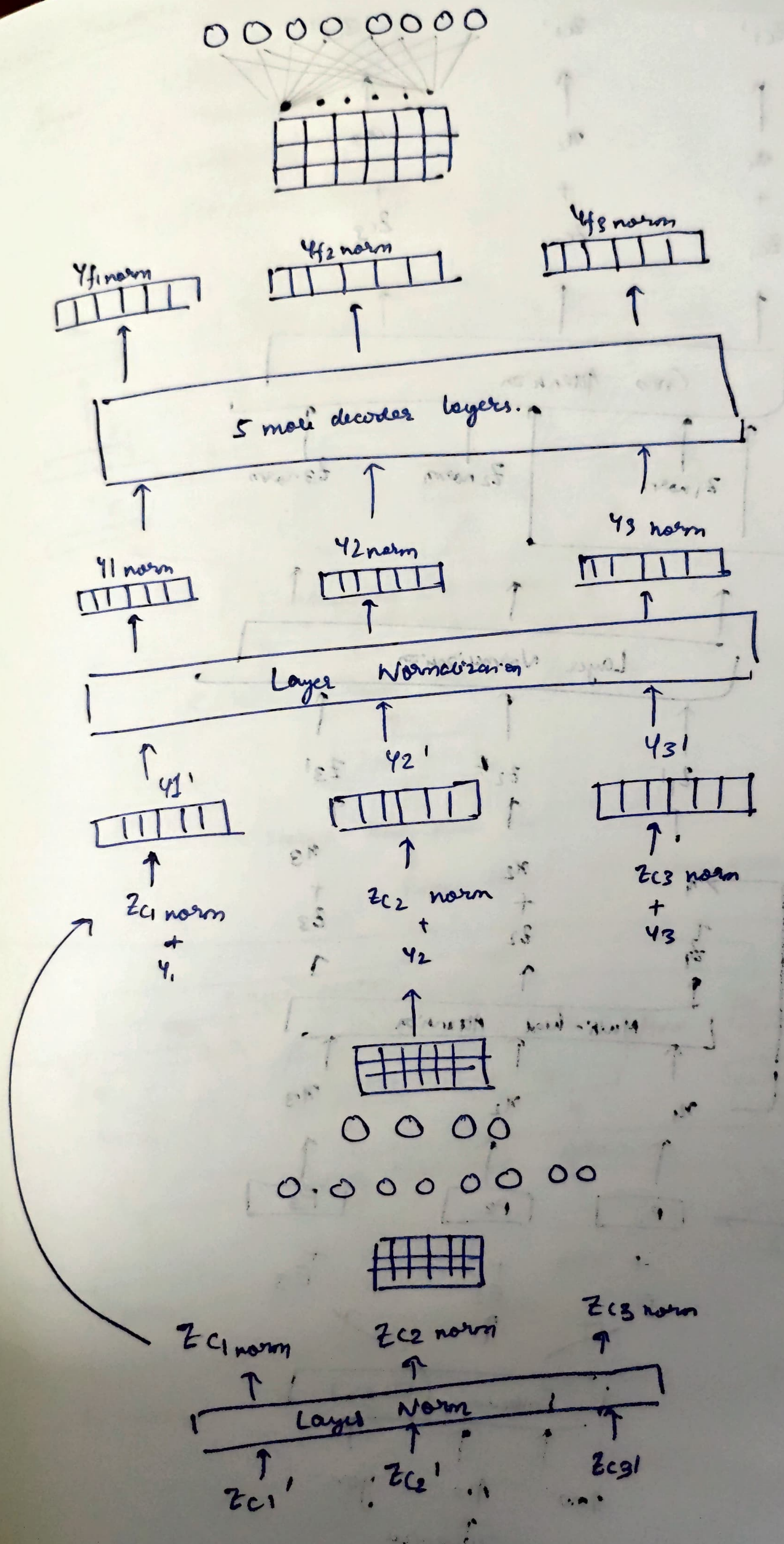
Encoder →



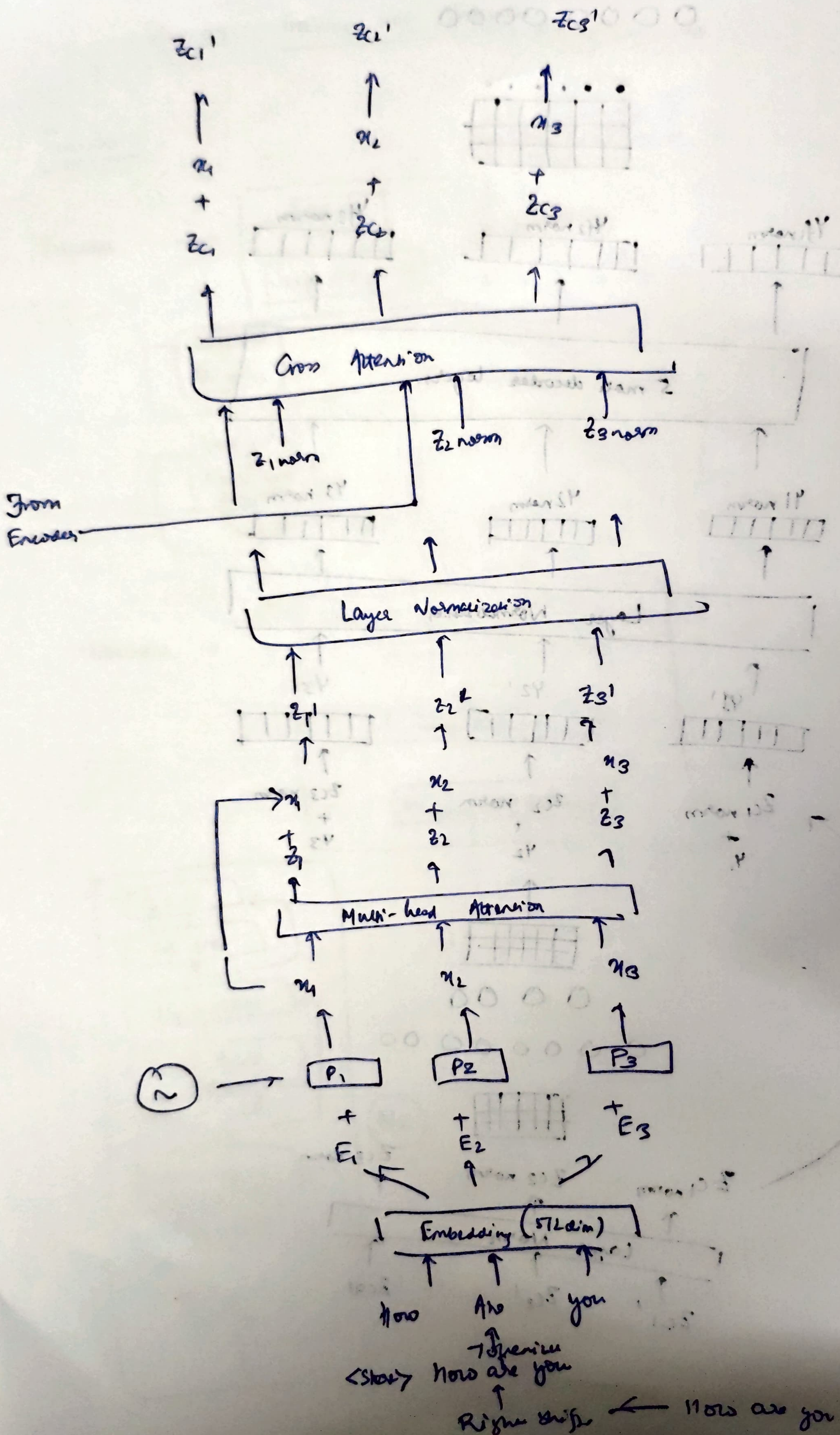
Decoder →







Yf1 norm
Yf2 norm
Yf3 norm
5 moli decoder layers.
Y1 norm
Y2 norm
Y3 norm
Layer Normalization
Y1'
Y2'
Y3'
Zc1 norm
+
Y1
Zc2 norm
+
Y2
Zc3 norm
+
Y3
Zc1 norm
Zc2 norm
Zc3 norm
Layer Norm
Zc1'
Zc2'
Zc3'



Zc1'
Zc2'
Zc3'
x1
x2
x3
+
Zc1
Zc2
Zc3
Cross Attention
Z1 norm
Z2 norm
Z3 norm
From Encoder
Layer Normalization
Z1'
Z2'
Z3'
x1
x2
x3
+
Z1
Z2
Z3
Multi-head Attention
x1
x2
x3
P1
P2
P3
+
E1
E2
E3
Embedding (512dim)
How
Are
you
<Start> How are you
Right shift
How are you

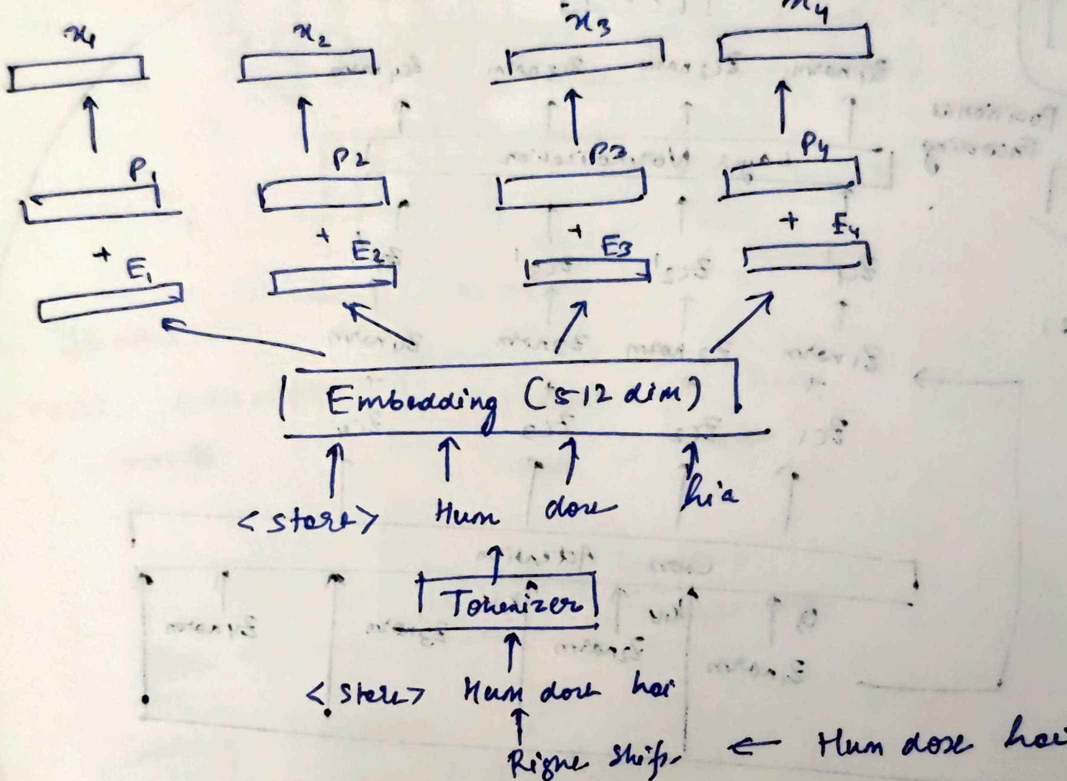
Machine Translation Task

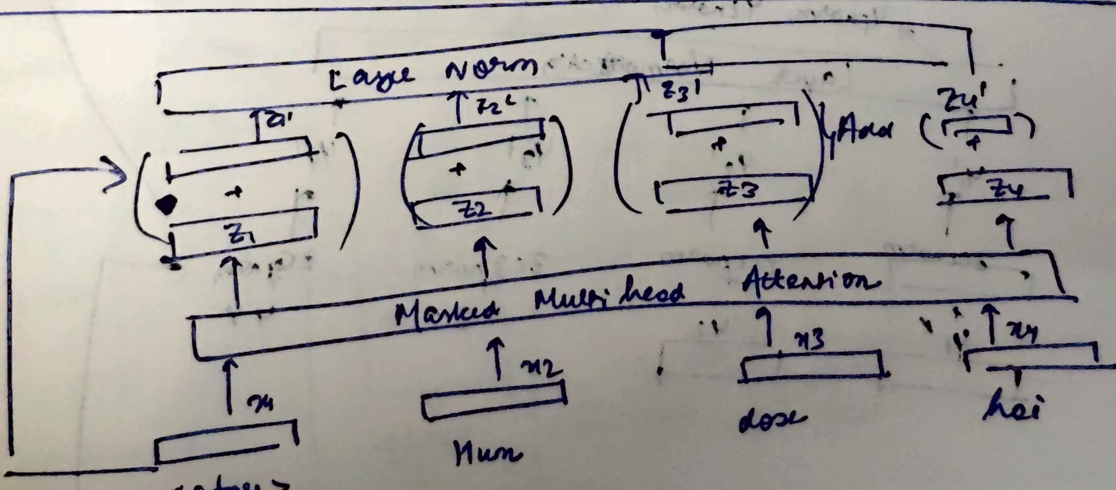
Input → 1. Shifting

2. Tokenization

3. Embedding

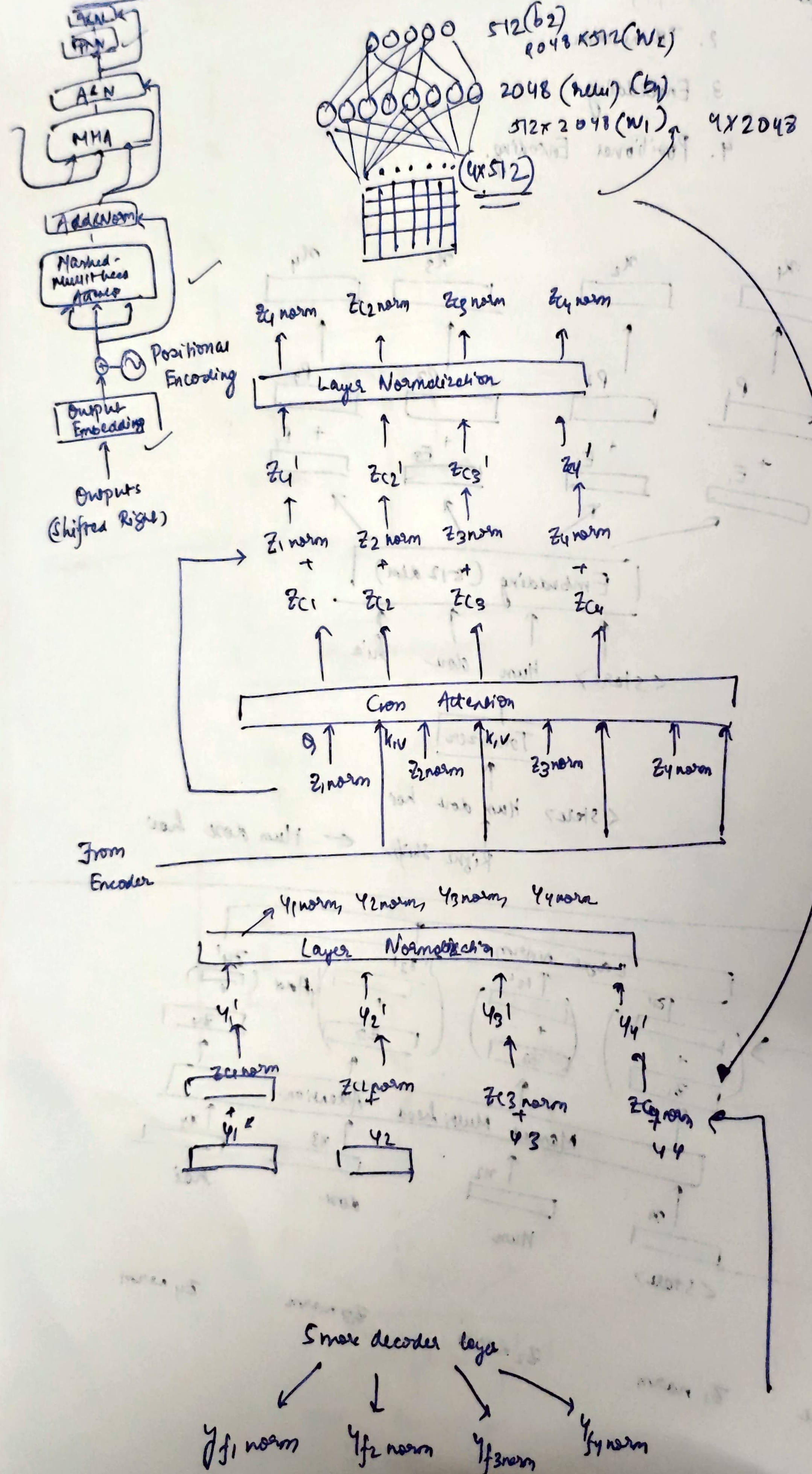
4. Positional Encoding.





we get z_1 norm z_2 norm z_3 norm z_4 norm.

Cross Attention Block

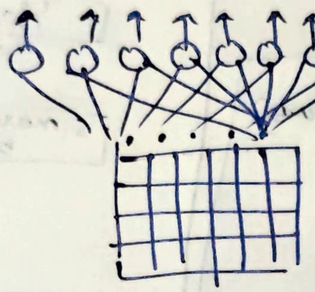


Eng	Hindi
I am good	Mai badhiya hu
We are friends	Hum dono hai

[V] = 10000

softmax ⇒ $\sum P_c = 1$ hum → 0.251

neurons ? → [V] → 10000



weights W_3 [512 x 10000]

Y_{f1} norm <start>
Y_{f2} norm hum
Y_{f3} norm dono
Y_{f4} norm hai

4 x 512

W_3 Y_{f} norm

(1 x 512) (512 x 10000)

(1 x 10000)

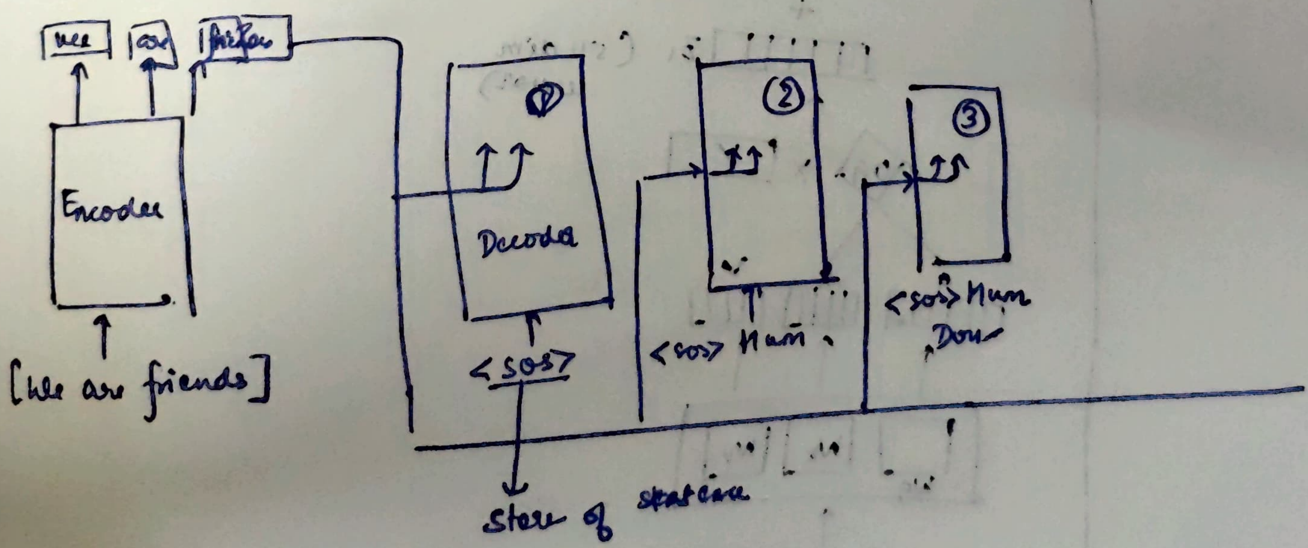
<start> → hum

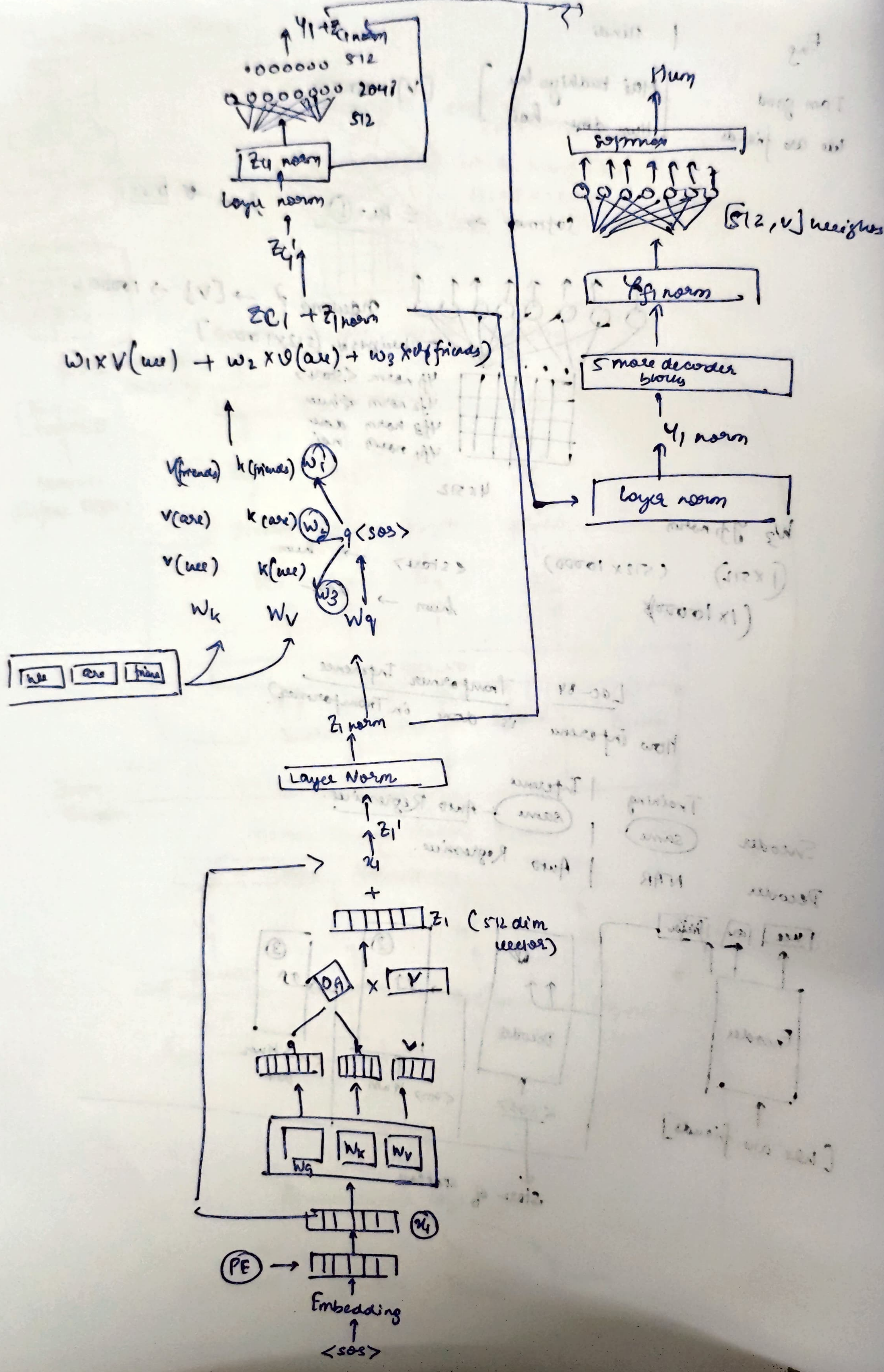
hum → dono

Lec-84 Transformer Inference

How inference is done on Transformer?

	Training	Inference
Encoder	same	same — Auto Regressive.
Decoder	NAR	Auto Regressive.





y1 + Z1 norm
512
2048
512
Z1 norm
Layer norm
Z1'
ZC1 + Z1 norm
W1 x V(we) + W2 x V(are) + W3 x V(friends)
V(friends)
k(friends)
W1
V(are)
k(are)
W2
q <sos>
V(we)
k(we)
W3
Wk
Wv
Wq
we
are
friends
Z1 norm
Layer Norm
Z1'
x1
+
Z1 (512 dim vector)
QA
x
V
q
k
v
Wq
Wk
Wv
x1
PE
Embedding
<sos>
Hum
softmax
[512, V] weights
Y1 norm
5 more decoder blocks
Y1 norm
Layer norm

→ Next Step → 2 times

x_1 ↓ <sos> x_2 ↓ Hum

	<sos>	Hum
<sos>	x	x → 0
Hum	x	x

→

0.2	0
0.1	0.6

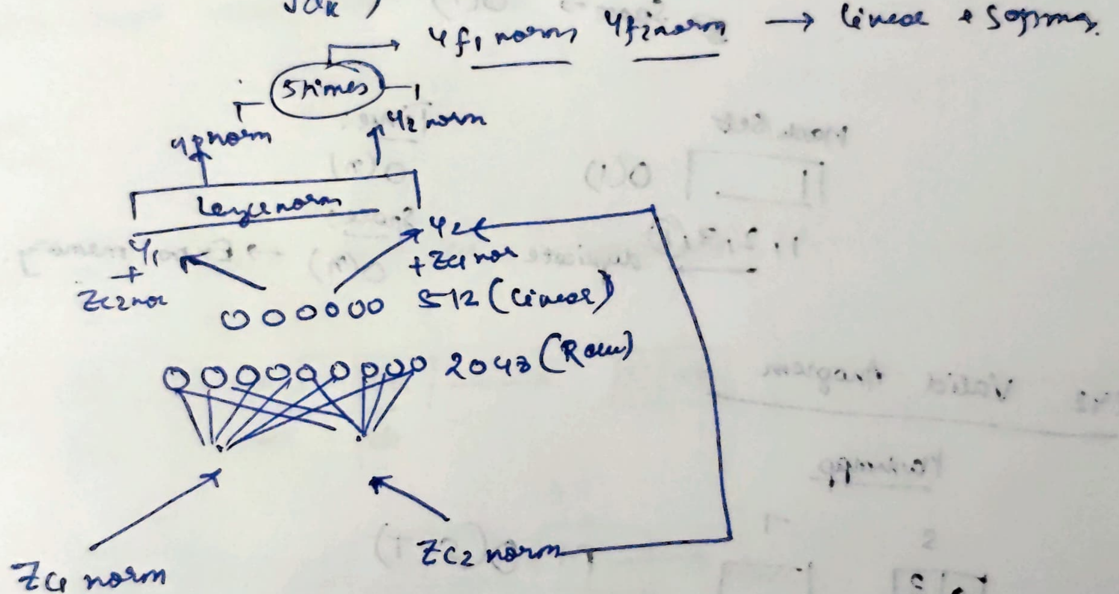
$$\frac{1}{\sqrt{d}}$$

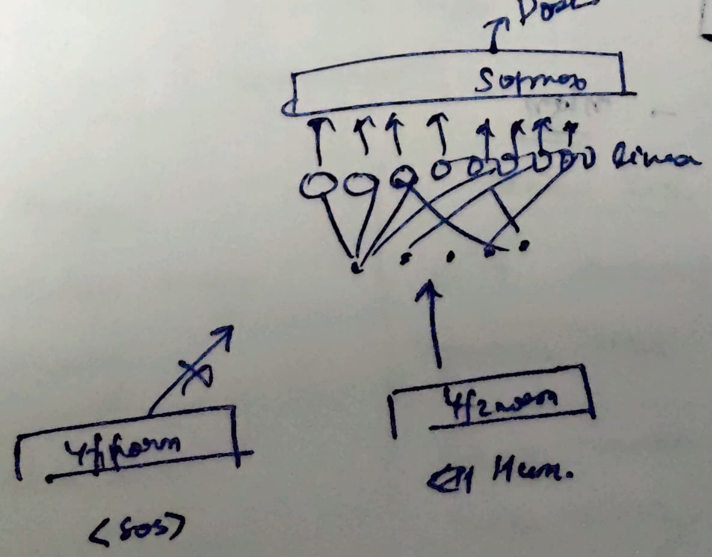
- Training → masking
- Inference → masking

masking

	sos	hum	dose
<sos>	x	0	0
hum	0	x	0
dose	x	x	x

$$\text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) \times V$$





· Increase tokens at every time step.